CLOs	PLOs
1, 2, 3, 4	1, 2, 4, 5,

Objectives:

This lab activity aims to enhance students' understanding and practical skills in SQL operations, focusing on essential commands such as

- **Join clause**
- **Inner Join**
- **Outer Join**
- **Cross Join**
- **Self-Join**

Tools:

- **MySQL (MySQL Workbench),** PostgreSQL, Microsoft SQL Server,
- Oracle, SQLite, Xampp

Exercise:

Below is a comprehensive case study for practicing SQL joins. Here we have a university database. This database will contain several tables, each representing different entities, such as departments, students, courses, and enrollments. The case study will be structured to encourage the use of various types of joins, including inner joins, left joins, right joins, cross joins, and self-joins.

2.1 Departments

department_id	department_name	department_head
1	Computer Science	Dr. Alice Johnson
2	Mathematics	Dr. Bob Smith
3	Physics	Dr. Carol White
4	Literature	Dr. David Green

2.2 Students

student_id	student_name	department_id	enrollment_year
101	John Doe	1	2021
102	Jane Smith	2	2020
103	Emily Davis	3	2022
104	Michael Brown	1	2019
105	Sarah Wilson	4	2023

2.3 Courses

course_id	course_name	department_id	credits
201	Introduction to CS	1	4
202	Data Structures	1	3
203	Calculus	2	4
204	Quantum Mechanics	3	5
205	English Literature	4	3
206	Algorithms	1	4

2.4 Enrollments

enrollment_id	student_id	course_id	semester	grade
301	101	201	Fall	A
302	101	202	Spring	B
303	102	203	Fall	A-
304	103	204	Spring	B+
305	104	206	Fall	A
306	105	205	Spring	B
307	101	203	Spring	B-
308	102	201	Fall	C+

2. Case Study Scenarios and Questions

The following questions are designed to help students practice different types of SQL joins:

1.) List all students along with the courses they have enrolled in.

2.) Find the names of students who have taken courses offered by the Computer Science department.

3.) Show the grades obtained by each student for each course in the Fall semester.

4.) List all students and the courses they have taken, including students who have not enrolled in any courses.

5.) Display all courses along with the names of students enrolled in them, even if a course has no students enrolled.

6.) List all enrollments, including details about students even if the course information is not available.

7.) Generate a list of all possible student-course pairs, even if a student is not enrolled in the course.

8.) Find pairs of students who are in the same department.

9.) Identify departments where multiple students enrolled in the same year.

10.) List the students who have taken courses outside their department and the grades they obtained.

11.) Find the department heads whose departments have the most students enrolled in courses for the Fall semester.

12.) Show the average grade per course for each department.

Marking Criteria:

Criteria	Marks	Details
Implementation of Joins (Inner, Outer, Cross, Self)	4	Correct application of different types of joins in queries.
Correctness of Queries	2	Queries correctly retrieve the required data without syntax or logical errors.
Handling of NULL values & Unmatched Data	1	Proper use of outer joins to include unmatched data where required.
Use of Aggregation & Filtering	2	Correct use of COUNT(), AVG(), and filtering conditions in queries.
Optimization & Readability	1	Queries are well-structured, efficient, and use appropriate aliases or formatting.